树与二叉树 5

章节脉络



树用父子关系组织层次。二叉树在此基础上区分左、右位置，遍历把结构转成一个处理次序；哈夫曼树、并查集和堆则各自增加约束，分别处理编码、集合合并和最值访问。它们共享树的表示工具，但各自的性质不能互相替代。

课程连接·目录树和进程树

操作系统的目录层次、进程派生关系可用树表达。硬链接若允许同一对象有多个上级入口，目录对象关系就不能直接套用“除根外唯一双亲”；先核对对象与链接的含义。

5.1 本章知识结构

- **树与森林的概念**：结点与父子兄弟；有序树与无序树；结点度与树度；层次高度与路径；边数及叶数计数。
- **二叉树性质与表示**：满与完全二叉树；叶数与双孩子结点；顺序编号及空位；二叉链表空域。
- **遍历与线索化**：先中后序及层序；非递归栈状态；遍历序列还原；中序线索与标志；前驱后继及边界。
- **树森林的存储与转换**：双亲表示；孩子表示；孩子兄弟表示；左孩子右兄弟转换；先根后根与二叉遍历。
- **哈夫曼树与编码**：带权路径长度；合并两个最小权根；最优性与编码非唯一；前缀编码与剩余容量；多路归并的零权补叶。
- **并查集与堆的连接**：双亲森林与代表元；按规模合并；路径压缩不改集合；完全性与堆序；插入上滤及删根下滤。

树的概念和计数性质是后续推导的基础。读遍历时同时追踪程序、栈或队列与输出序列；读编码和并查集时，再考察它们各自增加的约束。

5.2 树的结点、度、层次与路径

定义 5.2.1 (树及结点的度)

有序树规定同一结点各子树的先后，交换两个孩子可能改变树；**无序树**不区分这种先后。森林是零棵或多棵互不相交的树，删去一棵非空树的根后，各子树构成森林；在一片森林上加新根、把各棵树根接为孩子，又得到一棵树。普通树的有序性与“关键字从小到大”无关。

非空树有且仅有一个根。除根外每个结点恰有一个双亲，根到任意结点只有一条简单路径；树的各子树互不相交。一个结点的孩子数称为该结点的**度**，树的度是所有结点度的最大值。度为 0 的结点称为叶结点。树中“度”通常指数孩子的个数，与无向图把关联边条数称为度的约定不同。

最小反例·度为 2 未必是二叉树

最小情形只有根与一个孩子：普通无序树不区分这个孩子在左还是在右，二叉树却区分两种位置。因此“每结点至多两个孩子”尚未表达二叉树的全部结构语义。

本章按王道课件约定，根为第 1 层，结点的层次（深度）从根向下按层计，树的高度（深度）为最大层号；结点高度从该结点向下到最深叶子按层计，叶子高度为 1，空树高度为 0。根到结点的路径长度仍按边数计，等于该结点层次减 1。题目若另行按边数定义高度或深度，须先换算全部相关公式。

路径长度以边数计，不以沿途结点数计。树的路径长度是根到各结点路径长度的总和；树高由最长的一条路径决定，二者不同。孩子、子孙、兄弟也要区分：兄弟共有同一个双亲，与堂兄弟不是同一个概念。

5.2.1 例 5-1　怎样从非叶结点数求叶结点数

例 5.2.1 (配套案例)

【配套例题】一棵度为 4 的树，含 20 个度为 4 的结点、10 个度为 3 的结点、1 个度为 2 的结点、10 个度为 1 的结点。叶结点数为（　）。

A. 41　　B. 82　　C. 113　　D. 122

解析。设叶数为 n_0。一方面总结点数为 $n = n_0 + 20 + 10 + 1 + 10 = n_0 + 41$；另一方面每条边从一个双亲指向一个孩子，边数是度数总和 $4 \times 20 + 3 \times 10 + 2 + 10 = 122$。树有 $n - 1$ 条边，所以 $n_0 + 40 = 122$，得 $n_0 = 82$，答案为 **B**。

一般地，令 n_i 为度为 i 的结点数，由 $\sum_i n_i - 1 = \sum_i i n_i$ 得

$$n_0 = 1 + \sum_{i \ge 2} (i-1) n_i .$$

度为 1 的结点不出现在右端，是因为在一条边上再串入一个单孩子结点，结点数、边数各增加 1，叶数不变。因此只给定 n_2, n_3 和叶数，未必能确定全树结点总数；还可能插入任意多个度为 1 的结点。

每结点至多有 m 个孩子的树，第 i 层至多有 m^{i-1} 个结点，高度 h 至多容纳 $(m^h - 1)/(m-1)$ 个结点（$m > 1$）。求最小高度时，找使容量至少达到 n 的最小整数 h。例如至多 3 叉、50 个结点，高 4 时最多 40 个，高 5 时最多 121 个，所以最小高度 5。这里必须把“度为 3”理解为全树最大孩子数 3，不能要求每个非叶结点都恰有 3 个孩子。

5.2.2 度为 m 的树与 m 叉树

“度为 m” 要求全树至少有一个结点恰有 m 个孩子；“m 叉树” 只要求每个结点至多有 m 个孩子。二者都满足每层的容量上界，最少结点数却不同。对非空树，按根为第 1 层：

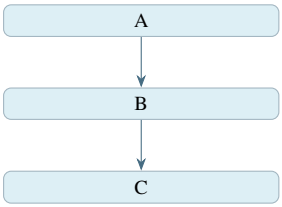
条件	最少结点数	最多结点数
高度 h 的 m 叉树	h，取一条单支链	$(m^h-1)/(m-1)$，$m>1$
高度 h 且度恰为 m	$h+m-1$，需存在 m 个孩子的分支	同上
有 n 个结点的 m 叉树	最小高度 $\lceil \log_m(n(m-1)+1) \rceil$	最大高度 n

第二行须有能容纳 m 个孩子的高度：$m>0$ 时 $h\geq 2$；单结点树度为 0。$m=1$ 时是一条链，容量公式改为 $n=h$，避免分母为 0。公式中的 “至多” 允许空缺，不能据容量上界推断某一棵给定树的实际形状。

例如高度 4、度为 3 的树至少有 $4+3-1=6$ 个结点：在一条 4 结点链的某个非叶结点处再增加 2 个叶即可；只说 “高度 4 的 3 叉树”，最少则只有 4 个结点。

错项排除·叶公式只用了二叉条件

证明 $n_0=n_2+1$ 时没有用到 “完全” 或 “满”。若选项声称任意二叉树还必有至多一个度为 1 结点，就偷偷增加了完全性；一条三结点单支链即可排除。



三结点单支链有 2 个度 1 结点；叶数公式仍成立。

5.3 二叉树、满二叉树与完全二叉树

二叉树的每个结点至多有两个孩子，而且左右有别。只有一个孩子时，左孩子与右孩子形成不同结构；二叉树也不是简单地把 “度不超过 2 的普通树” 换个名字。普通树的单孩子若不规定位置，就不区分左、右这两种形态。

高度为 h、每一层都达到最大结点数的二叉树称为**满二叉树**，共有 2^h-1 个结点。**完全二叉树**按满二叉树从上到下、从左到右编号，已有结点恰占前 n 个编号。最后一层可以不满，但只能缺右侧连续的一段；其余各层则必须排满。

“每个非叶结点恰有两个孩子” 只说明不存在度为 1 的结点，未必是这里定义的满二叉树。叶结点可能分布在不同层。使用外文资料中的 full、perfect 等名称时，也要先看定义。

二叉树第 i 层至多 2^{i-1} 个结点。非空二叉树有 $n_0=n_2+1$，由上一节公式直接得到。高度为 h 时，结点数满足 $h\leq n\leq 2^h-1$；给定 n，最小高度为 $\lceil \log_2(n+1) \rceil$，最大为 n。例如 50 个结点，最小高度 6。

完全二叉树有 $\lfloor n/2 \rfloor$ 个非叶结点、$\lceil n/2 \rceil$ 个叶结点，最多只有一个度为 1 的结点，且它只能有左孩子。普通二叉树不能沿用这些完全性结论。

定理 5.3.1 (二叉树的叶与双孩子结点)

任意非空二叉树中，若 n_i 表示度为 i 的结点数，则 $n_0=n_2+1$。

按结点与按孩子数边

结点总数 $n=n_0+n_1+n_2$，边数一方面为 $n-1$，另一方面为 n_1+2n_2。令两式相等并消去 n_1，即得 $n_0=n_2+1$。证明只需二叉树，不要求完全、满或平衡。

□

5.4 顺序编号与链式表示

不变量·编号表达位置而非秩

顺序存储中下标表达满二叉树的位置，删除缺失位置后不能把剩余元素紧凑前移而继续沿用 $2i, 2i+1$。链表遍历得到的次序也不是这种位置编号。

完全二叉树从 1 开始编号，结点 i 的双亲为 $\lfloor i/2 \rfloor$，左、右孩子为 $2i, 2i+1$，但只有编号不超过 n 的位置才存在。若下标从 0 开始，孩子公式改为 $2i+1, 2i+2$，非根结点的双亲为 $\lfloor (i-1)/2 \rfloor$。

任意二叉树也可以按满二叉树位置编号，但缺失的位置必须留空。因此实际结点少，顺序存储占用的空间未必小。高度 h 的一条全右链，从 1 开始编号时最后编号为 $2^h - 1$，只存 h 个结点却可能需要指数级下标范围。

5.4.1 例 5-2　任意形态的顺序存储需要预留多少

例 5.4.1 (配套案例)

【配套例题】对于任意一棵高度为 5、有 10 个结点的二叉树，采用顺序存储结构保存，每个结点占 1 个存储单元，只存放结点数据信息。存放该二叉树需要的存储单元数量至少为（　）。

A. 31　　B. 16　　C. 15　　D. 10

解析。本题要求容量能够保存这种约束下的任意形态。高度 5 的满二叉树位置编号到 31；合法的 10 结点形态完全可能包含第 31 号位置（保留通往该位置的祖先，再安排其余结点）。因此为保证任意形态可保存，至少预留 31 个位置，答案为 **A**。并非每一棵给定的 10 结点树都实际用到下标 31；不能把 “保证任意形态” 读成 “在所有形态中挑一个最省空间的”。

二叉链表为每个结点保存左、右孩子指针。n 个非空结点共有 $2n$ 个孩子指针域，真正指向孩子的有 $n-1$ 个，所以空域为 $n+1$ 个。若再增加双亲指针，根的双亲域为空，其他 $n-1$ 个双亲域非空；总空域为 $3n - 2(n-1) = n+2$。某个结点被多少指针指向，不只由它自己有几个孩子决定：孩子的双亲域也会指回它。

5.4.2 二叉链表的教材定义

```c
typedef struct BiTNode {
    ElemType data;
    struct BiTNode *lchild, *rchild;
} BiTNode, *BiTree;
```

`BiTree T` 本质上是根结点指针，空树为 `T==NULL`。`T->lchild`、`T->rchild` 是孩子指针，不能把一个 `BiTNode` 对象和指向它的 `BiTree` 混用。下文 `visit(T)`表示处理该结点的数据；具体打印、累计或复制操作由应用决定。

顺序存储中，合法孩子编号还必须对应一个非空位置；对于有空洞的普通二叉树，`2*i<=数组容量`只说明下标未越界，并不保证该孩子实际存在。

5.5 遍历、还原与树形递归

最小反例·先序加后序的歧义

最小反例为 A 与其唯一孩子 B：B 在左或在右，先序都是 AB，后序都是 BA。缺少中序时，序列没有给出这个单孩子的左右位置；重复值还会带来身份歧义。

先序、中序、后序分别把“访问根”的位置放在左、右子树递归的前、中、后；三个次序都先处理左子树再处理右子树。对下图：

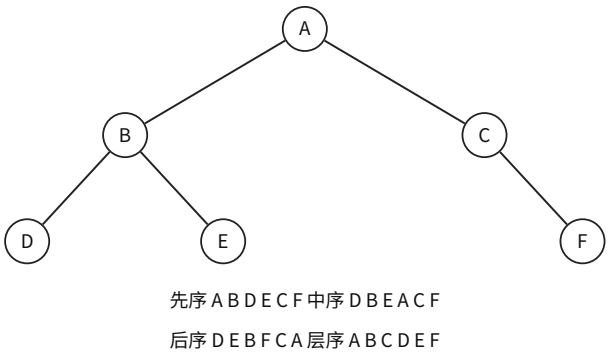


Figure 5.1: 同一棵二叉树的先中后序及层序；空的左孩子不应被当作另一个结点

先序为 `A B D E C F`；中序为 `D B E A C F`；后序为 `D E B F C A`；层序为 `A B C D E F`。先序在深入子树之前处理结点，适合复制或输出结构；后序在左右结果齐备后处理根，适合求子树大小、高度、释放结点。访问位置应服从依赖，不能只因为某种遍历最常见就选它。

```
int TreeDepth(BiTree T) {
    if (T==NULL) return 0;
    int hl=TreeDepth(T->lchild), hr=TreeDepth(T->rchild);
    return 1+(hl>hr ? hl : hr);
}
```

这段代码每个结点只处理一次，时间 $O(n)$；调用栈深为 $O(h)$。求高度时，叶结点返回 1，与本章高度约定一致。交换所有左右子树可以递归地先交换两个子树内部，再交换两个孩子指针，从而把中序序列整体反向；交换不改变根的身份，也不改变哪些结点是叶结点。

课件中的另一棵树增加了 D 的右孩子 G、C 的左孩子 F，可同时检查左右位置、访问时机与输出次序：

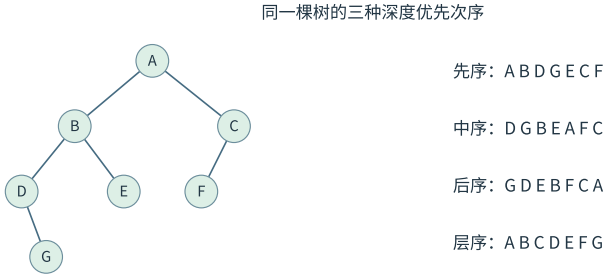


Figure 5.2: 原树与先序、中序、后序、层序的对应序列

5.5.1 递归遍历的三种访问位置

条件辨析·访问位置

先中后序共用左右递归骨架，根的访问位置分别在左递归前、两次递归间、右递归后。依赖子树结果的操作宜在返回后处理。

```
void PreOrder(BiTree T) {
    if (T!=NULL) {
        visit(T);
        PreOrder(T->lchild);
        PreOrder(T->rchild);
    }
}
void InOrder(BiTree T) {
    if (T!=NULL) {
        InOrder(T->lchild);
        visit(T);
        InOrder(T->rchild);
    }
}
void PostOrder(BiTree T) {
    if (T!=NULL) {
        PostOrder(T->lchild);
        PostOrder(T->rchild);
        visit(T);
    }
}
```

把每个递归结点看成有三个停留点：进入左子树前、左子树返回后、右子树返回后。三种遍历仅改变 `visit` 所在停留点，递归下降和返回的骨架相同。时间都是 $\Theta(n)$，最大同时存活的调用数由树高决定，辅助栈为 $O(h)$；单支树可达 $O(n)$，平衡形态为 $O(\log n)$。表达式树的先序、后序分别产生前缀、后缀表达式；中序输出中缀表达式时要根据结构添加括号，才能保留原运算顺序。

5.5.2 先序加中序为什么能还原

假设结点标识互不相同。先序第一个元素为根，在中序中找到根后，左侧全属于左子树，右侧全属于右子树；由左子树的长度，能把先序余段分成左右两段，再递归。上图根为 A，中序左段 `D B E` 长 3，故先序左段是 `B D E`，根 B 又把它分成 D 和 E。

中序加后序也能唯一还原，根取后序最后一个；先序加后序一般不能区分一个单孩子在左还是在右。只给含 n 个不同标识的先序序列，形态数仍为卡特兰数 C_n，例如 4 个元素为 14 种。结点值重复时，“在中序中找到根的位置” 可能不唯一，前述唯一性需要额外身份信息。

5.5.3 由层序和中序还原

结点标识互异时，层序首项是根；中序在根处分开左右子树。把其余层序项按 “是否属于左侧中序集合” 稳定筛分，得到左右子树各自的层序，再递归。

课件例：层序 `A B C D E`、中序 `A C B E D`。根为 A，中序左侧为空，所以 A 没有左子树；右子树层序为 `B C D E`，中序为 `C B E D`。该子树根 B，左子树只有 C；右子树根 D 且左孩子为 E。最终为 `A(空,B(C,D(E,空)))`。由此得到先序 `A B C D E`、后序 `C E D B A`。

课件另一组先序 `D A E F B C H G I`、中序 `E A F D H C B G I`，根 D 将中序分为 `E A F` 与 `H C B G I`；左子树根 A，孩子 E、F；右子树根 B，左子树为 C 且左孩子 H，右子树根 G 且右孩子 I。逐

层分段比仅凭序列外观猜树形可靠。

条件辨析·层序先分根再筛子树

层序中同一子树的相对次序保留，中序负责划分身份集合。不能只按层序剩余部分的长度直接切左右两段。

已知序列，标识互异	一般能否唯一还原	决定子树范围的依据
先序＋中序	能	先序首项定根，中序分左右
后序＋中序	能	后序末项定根，中序分左右
层序＋中序	能	层序首项定根，中序分左右并筛层序
先序＋后序	一般不能	单孩子的左右位置无法区分
先序＋层序、后序＋层序	一般不能	两结点的单左、单右树即为反例

“需要中序” 是一般二叉树、仅按这些遍历组合还原时的结论；增加满二叉树、显式空指针标记等约束后，要按新增信息重新分析。

非递归中序遍历用显式栈保存未完成的祖先：沿左链入栈，到空位置时弹栈访问，再转到其右子树。上图先依次压入 A、B、D，弹出 D 后没有右孩子，接着弹出 B 并转到 E；E 处理完才回到仍在栈中的 A。

```
inorder(root):
    p=root; 建空栈 S
    while p 非空 or S 非空:
        while p 非空:
            push(S,p); p=p->lchild
        p=pop(S); visit(p)
        p=p->rchild
```

非递归后序还需分辨右子树是否已经处理，可保存最近访问结点 `last`。从左子树返回时先看栈顶；它有尚未处理的右子树，就进入右子树，不能立即弹根。

```
postorder(root):
    p=root; last=NULL; 建空栈 S
    while p 非空 or S 非空:
        if p 非空:
            push(S,p); p=p->lchild
        else:
            q=top(S)
            if q->rchild 非空 and last!=q->rchild:
                p=q->rchild
            else:
                visit(q); last=pop(S)
```

`last` 只在一次完整子树处理后更新，因此它等于 `q->rchild`，表示右子树根已经最后访问过。每个结点各入栈、出栈一次，时间仍为 $O(n)$，栈最多同时保存一条祖先路径，空间为 $O(h)$。

层序遍历把根入队，反复出队访问并依次入队左右孩子。判定完全二叉树可在层序遍历中设置 “已经遇到空孩子” 标志：一旦出现空位置，此后的孩子位置不得再出现非空结点。只排除 “有右孩子而无左孩子” 还不够，还要排除遇到首个空孩子位置后又出现非空孩子的情况；一个结点只有左孩子，本身未必违反完全性。

5.5.4 例 5-3　中序前驱和后继在哪一侧

例 5.5.1 (配套案例)

【配套例题】p、q、v 都是二叉树 T 中的结点，v 有两个孩子，T 的中序遍历序列形如 …,p,v,q,…。正确的是（　）。

A. p 没有右孩子，q 没有左孩子　B. p 没有右孩子，q 有左孩子

C. p 有右孩子，q 没有左孩子　D. p 有右孩子，q 有左孩子

解析。v 有左子树，紧邻它之前的 p 必在左子树的最右端；若 p 还有右孩子，应先访问那棵右子树，p 就不可能紧邻 v。对称地，q 是右子树的最左端，不可能有左孩子。答案为 **A**。这里的“最右端”指沿右孩子指针一直走到不能再走的位置，不能按图上哪个结点画得最靠右来判断。

遍历问题先固定三个条件

先写清访问根的位置、左右孩子顺序、空树处理。还原问题再检查结点身份是否唯一；若给先序与中序，根由先序首项决定，子树边界由中序决定，不能按数值大小猜左右。

层序遍历的队列表示“已发现但尚未处理”的结点。以本章图为例，A 出队后 B、C 入队，接着 B 出队才把 D、E 加入队尾，所以 C 在 D 之前处理。

```
levelOrder(root):
    if root为空: return
    Q入队root
    while Q非空:
        p=Q出队; visit(p)
        if p.lchild非空: Q入队p.lchild
        if p.rchild非空: Q入队p.rchild
```

时间 $O(n)$；队列峰值由树的宽度决定，最坏 $O(n)$。不能把递归遍历的 $O(h)$ 调用栈结论搬到层序队列。

错项排除·先看 tag 再沿指针

线索填写后，一条非空指针可能指向祖先或兄弟方向的前后继。若忽略 tag 把它当孩子继续递归，就可能绕回已访问位置；“指针非空”不等价于“存在子树”。

5.6 线索二叉树：区分孩子链接与遍历线索

二叉链表有许多空孩子指针。把空左域改记某遍历次序下的前驱，把空右域改记后继，便得到相应的**线索二叉树**。新增 `ltag`、`rtag` 区分该域是孩子还是线索；本书约定 0 表示孩子，1 表示线索。寻找子树时必须先看标志，不能沿一条线索继续递归，否则可能回到祖先形成循环。

中序线索树中，若结点右域为线索，可直接到中序后继；若右域为孩子，后继是右子树中沿真实左孩子链到达的第一个结点。前驱同理：左域为线索则直接取，否则到左子树最右端。图中每个结点的 L、R 分别表示左、右指针域，其后数字为对应的 `ltag`、`rtag`。E 的 R1 指向 A，C 的 L1 也指向 A；两条线索各有独立的起点和箭头落点。D 的 L1 和 F 的 R1 则分别接空前驱、空后继端点，本图采用无头结点、首尾取空的约定。

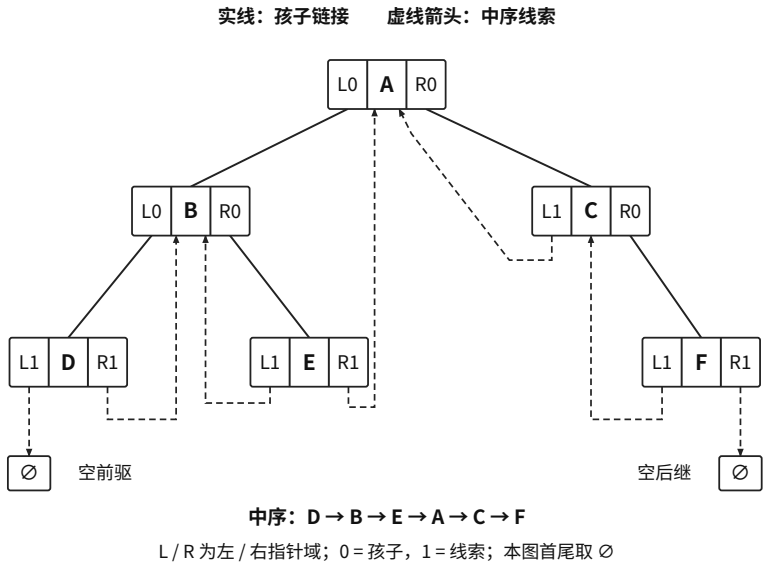


Figure 5.3: 虚线为中序线索，实线为孩子；首尾边界单独标出

$n+1$ 个原空孩子域均可改为线索域，其中首结点的前驱和末结点的后继需约定为空或指向专设头结点。题目说 “线索数为 $n+1$” 时通常统计这些线索域，包含首尾边界；若只统计实际指向普通结点的线索，则边界两项另算。加头结点后，其额外指针也不应混入原树的 $2n$ 域计数。

构造中序线索时，用 `pre` 保存刚访问过的结点，初值为空。处理当前结点 p 时，若其左域为空，填入 pre 并设置左线索标志；若 pre 的右域为空，填入 p 并设置右线索标志，随后令 pre=p。递归只沿真实孩子执行，避免把刚填的线索又当作子树。

错项排除·先看 tag 再沿指针

线索填写后，一条非空指针可能指向祖先或兄弟方向的前后继。若忽略 tag 把它当孩子继续递归，就可能绕回已访问位置；“指针非空” 不等价于 “存在子树”。

```c
typedef struct ThreadNode {
    ElemType data;
    struct ThreadNode *lchild, *rchild;
    int ltag, rtag;  // 0：孩子；1：线索
} ThreadNode, *ThreadTree;

void InThread(ThreadTree p, ThreadTree &pre) {
    if (p!=NULL) {
        InThread(p->lchild, pre);
        if (p->lchild==NULL) {
            p->lchild=pre;
            p->ltag=1;
        }
        if (pre!=NULL && pre->rchild==NULL) {
            pre->rchild=p;
            pre->rtag=1;
        }
        pre=p;
        InThread(p->rchild, pre);
    }
}
void CreateInThread(ThreadTree T) {
    ThreadTree pre=NULL;
    if (T!=NULL) {
        InThread(T, pre);
        pre->rchild=NULL;
        pre->rtag=1;
    }
}
```

条件辨析·中序末结点的右域

中序最后访问的是最右结点，它的右孩子必为空；后序最后访问的是根，根却可能有右孩子。收尾语句适用的遍历不能混用。

调用前所有原孩子标志为 0。图中先访问 D，其空左域指向空前驱；随后访问 B，D 的空右域才补为指向 B 的线索。访问 E 时填左线索指向 B，访问 A 时再把 E 的空右域补为 A。遍历结束后，pre 为最后结点 F，将其空右域标为后继线索，线索目标按约定为空或为头结点。若使用头结点，首结点空前驱也应统一改指头结点。

先序、后序线索的前驱后继依照各自次序，不可沿用中序规则。后序中，若叶结点 X 存在左兄弟 Y，则 X 是双亲的右孩子；左子树和右子树都处理完后才访问双亲，因此 X 的后序后继是双亲，而不是左兄弟。一般后序线索树若无双亲信息，不一定能像中序那样方便地只凭孩子和线索完成全部后继访问。

5.6.1 线索树的访问算法与六种前驱后继

`pre` 以引用传入，使左子树递归更新后的 “刚访问结点” 能够传回当前层；若按值传指针，调用返回后当前层仍拿着旧值。教材中序线索化只对原始、标志均为 0 的二叉链表调用。右递归开始前，当前结点原本非空的右孩子未被改写；因此该版本可直接递归右孩子。再次对已线索化的树调用时，必须按标志辨别真实孩子。

```
ThreadNode *Firstnode(ThreadNode *p) {
    while (p->ltag==0) p=p->lchild;
    return p;
}
ThreadNode *Nextnode(ThreadNode *p) {
    if (p->rtag==0) return Firstnode(p->rchild);
    return p->rchild;
}
void InOrderThread(ThreadNode *T) {
    if (T==NULL) return;
    for (ThreadNode *p=Firstnode(T); p!=NULL; p=Nextnode(p))
        visit(p);
}
```

条件辨析·中序末结点的右域

中序最后访问的是最右结点，它的右孩子必为空；后序最后访问的是根，根却可能有右孩子。收尾语句适用的遍历不能混用。

`Firstnode` 的前提是参数非空、所有空左域均已正确标成线索；`Nextnode` 接收一个实际结点。这样无需递归栈即可中序遍历。找一次后继最坏可能沿一条左链，整趟遍历每条孩子链接只走常数次，故总时间 $O(n)$、额外空间 $O(1)$。

线索类型	找前驱	找后继
中序	左域为线索则直接取；否则左子树最右结点	右域为线索则直接取；否则右子树最左结点
先序	左域为线索则直接取；有左孩子时通常还需双亲信息	有左孩子先取左孩子，否则取右域（孩子或线索）
后序	有右孩子先取右孩子，否则取左域（孩子或线索）	右域为线索则直接取；有右孩子时通常还需双亲信息

先序有左孩子的结点，其前驱在子树之外；后序有右孩子的结点，其后继在子树之外，因此普通二叉线索链表未必能直接取得。若有双亲域：先序中，左孩子的前驱是双亲；右孩子无左兄弟时前驱是双亲，有左兄弟时为左兄弟子树最后一个先序结点。后序对称：右孩子的后继是双亲，左孩子无右兄弟时后继是双亲，有右兄弟时为右兄弟子树第一个后序结点。

先序线索化在访问根后才递归，刚建立的左线索可能指回祖先，故递归左侧必须检查 `ltag==0`。后序先完成两棵原子树再建线索，注意结束后末结点是根，根未必原本没有右孩子；处理收尾时应判断空右域，不能照搬中序 “最后结点右孩子必空” 的条件。

5.7 普通树的三种存储

双亲表示法为每个结点保存双亲的下标，根用 -1 等特殊值标识。找双亲 $O(1)$，找全部孩子通常需扫描整个数组。它适合从结点反复向根追踪，却不便于列举孩子。

孩子表示法为每个结点保存孩子链表，便于依次访问所有孩子；反查双亲时若没有另外保存双亲信息，仍需搜索。**孩子兄弟表示法**只保存两个链接：`firstChild` 指向第一个孩子，`nextSibling` 指向下一个兄弟。它能用两个指针表示孩子数不定的树，却不能把 `nextSibling` 理解成普通树的孩子。

选择表示时，应先看主要方向：查双亲、枚举孩子或遍历同层兄弟，分别依赖不同的链接。多保存一个双亲域会增加空间，但上溯不必

再搜索，代价降低。

不变量·转换不改变数据结点

孩子兄弟转换保留全部数据结点，但改变链接解释。原树根无兄弟，所以转换后二叉根无右孩子；森林的多个根由右链串起，不能把右链误认成原树的后代。

5.7.1 三种树存储的教材结构

```c
#define MAX_TREE_SIZE 100
typedef struct {
    ElemType data;
    int parent;
} PTNode;
typedef struct {
    PTNode nodes[MAX_TREE_SIZE];
    int n;
} PTree;

typedef struct CSNode {
    ElemType data;
    struct CSNode *firstchild, *nextsibling;
} CSNode, *CSTree;
```

双亲数组中根的 `parent=-1`，其余值为双亲在数组中的下标，不能把它当作元素数据。孩子表示法则在顺序顶点表的每个元素中保存孩子链首指针，链结点保存孩子下标和下一孩子指针；它是顺序与链式存储的结合。森林的双亲表示可有多个 `parent=-1`，孩子表示则须保存各根的位置。孩子兄弟表示中，森林各根沿 `nextsibling` 相连。

5.8 树、森林怎样转换为二叉树

采用“左孩子、右兄弟”约定：普通树第一个孩子成为二叉树左孩子，后续兄弟依次以右孩子相连。原树的根没有兄弟，故其对应二叉树的根没有右孩子。森林的各棵树根互为兄弟，再沿右链连接，第一棵树的根成为二叉树的根。

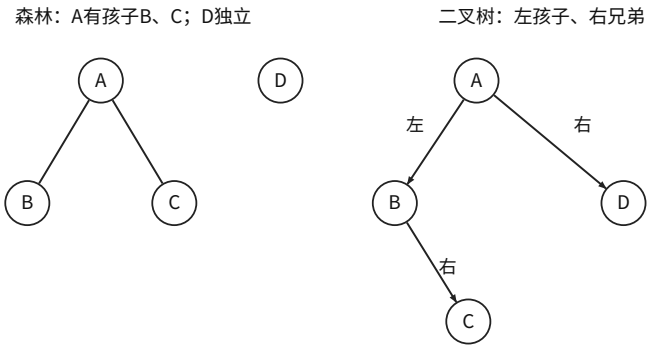


Figure 5.4: 普通树的孩子关系转成左孩子，兄弟关系转成右孩子

逆转换时，从某结点的左孩子出发沿右链走，沿途结点就是它的全部孩子；根及其右链上的各结点分别作为森林各棵树的根。转换改变了“左右链接的解释”，没有增加或删除数据结点。森林有 4 棵树、第一棵含 a 个结点时，二叉树根的左子树只含第一棵树除根外的 $a-1$ 个结点，其他树由根的右子树表示。

5.8.1 例 5-4　二叉树隔两条链接，在森林中是什么关系

例 5.8.1 (配套案例)

【配套例题】将森林转换成对应二叉树。若二叉树中 u 是 v 的父结点的父结点，则原森林中 u 与 v 可能的关系是（　）。

I. 父子关系　II. 兄弟关系　III. u 的父结点与 v 的父结点是兄弟关系

A. 只有 II　B. I 和 II　C. I 和 III　D. I、II 和 III

解析。从 u 到 v 恰经过两条二叉树链接。左—右表示 “先到 u 的第一个孩子，再到其下一个兄弟”，v 仍是 u 的孩子，I 可能；右—右表示连续两个兄弟链接，u、v 互为兄弟，II 可能。左—左表示祖孙，右—左表示 u 的兄弟的孩子，它们都不能产生 III 所说的父结点关系。因此答案为 **B**。应按每条链接的语义推回森林，不能把二叉树的双亲直接当作原树的双亲。

课程连接·编码和外排使用同一棵树

哈夫曼叶权可表示字符频数，也可表示外部归并段长。每次参加合并使该叶再负担一次权值，故总合并长度等于 WPL；第 8 章的最佳归并由此得到。

5.9 树与森林的遍历怎样对应

普通树的先根遍历先访问根，再依次先根遍历各子树；后根遍历先按顺序遍历各子树，再访问根。采用孩子兄弟转换后，树的先根次序对应二叉树先序，树的后根次序对应二叉树中序。原因是二叉树左链接深入第一个孩子，而右链接串联后续兄弟。

森林先序依次先根遍历各棵树，对应二叉树先序；森林中序先按中序处理第一棵树的子树森林，再访问该树根，最后处理其余树，对应二叉树中序。森林遍历要按定义递归到各棵树，并非在整片森林开头统一访问一次根。

层次遍历普通树时，出队一个结点后，按孩子链的次序把它的所有孩子入队；兄弟链接只用于枚举孩子，不能把整条兄弟链当成下一层的结点序列。

5.10 哈夫曼树、WPL 与前缀编码

给每个叶结点一个权值 w_i，根到它的路径长度（边数）为 d_i，**带权路径长度**为 $\mathrm{WPL}=\sum_i w_i d_i$。权值通常为频数或概率，内部结点不直接加入这项 “叶权乘深度” 的求和。求给定叶权下最小 WPL 的二叉树，可每次选当前两个最小权值合并为新结点，再把权值和放回候选集合，直到仅剩根。

为什么合并两个最小值：在一棵最优树中，可以让最深的一对兄弟叶承载两个最小权值而不增大 WPL；把它们收缩成一个权值为和的叶后，剩余结构仍应最优，否则替换后能得到 WPL 更小的树。每次合并都使这两棵子树中所有叶的深度增加 1，因此合并权值之和也等于最终 WPL。

5.10.1 例 5-5　加权平均码长怎样从合并过程得到

例 5.10.1 (配套案例)

【配套例题】6 个字符出现频次分别为 3、4、5、6、8、10，为它们构造哈夫曼编码，加权平均长度为（　）。

A. 2.4　　B. 2.5　　C. 2.67　　D. 2.75

错项排除·最优不意味着唯一

左右子树交换不会改变叶深和 WPL；同权候选的选择也可产生不同树。应排除 “不同编码必有不同平均码长”，同时避免把哈夫曼最优性扩大成完全或平衡。

解析。每轮重新从当前候选中取最小的两个，不能把原始序列分成相邻对后一次配完。

本轮合并	新权值	合并后候选权值
3 与 4	7	5、6、7、8、10
5 与 6	11	7、8、10、11
7 与 8	15	10、11、15
10 与 11	21	15、21
15 与 21	36	36

WPL 为 $7+11+15+21+36=90$，频次总和 36，所以平均码长为 $90/36=2.5$ 位/字符，答案为 **B**。不能把总码长除以 6，因为各字符出现频次不同。

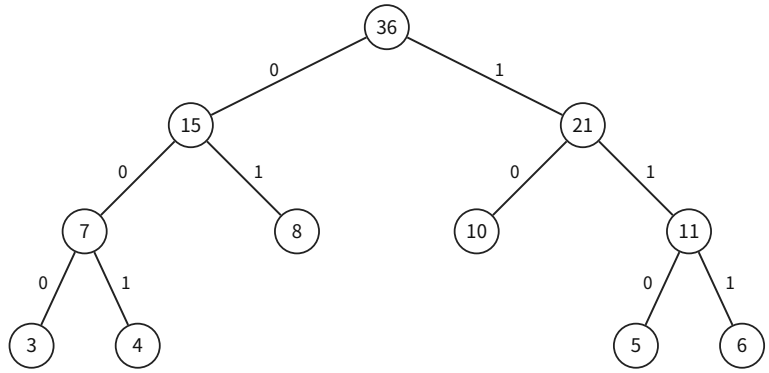


Figure 5.5: 同一合并过程构造的哈夫曼树；左右分支的 0、1 可对换

沿根到叶的 0、1 路径形成码字。字符只放在叶上，所以任何字符码都不是另一个字符码的前缀，这称为**前缀编码**。编码可因相等权值的选取、左右分支交换而不唯一，WPL 仍可相同。哈夫曼树没有度为 1 的内部结点，若有 n 个叶，内部结点 $n-1$、总结点 $2n-1$；它未必完全、未必平衡，n 个叶的最大高度可达 n。

5.10.2 王道课件中的合并与压缩实例

权值为 `1,3,2,2,7` 时，取相等权值的一种合法顺序：先合并 1 与 2 得到 3，再将另一个 2 与新 3 合为 5，原 3 与 5 合为 8，最后 7 与 8 合为 15。WPL 等于全部合并权值之和：$3+5+8+15=31$。权相等时可以换选，形态和码字可能变化，最优 WPL 保持 31。

再看课件的 100 题答案：C、A、B、D 分别出现 80、10、8、2 次。固定 2 位编码共需 200 bit；哈夫曼合并为 $2+8=10$、$10+10=20$、$20+80=100$，WPL 为 $10+20+100=130$ bit。按图中左 0 右 1，可取

C 为 0、A 为 10、D 为 110、B 为 111，平均码长 $130/100 = 1.3$ bit/题。与固定 2 位相比，编码数据长度为原来的 65%，节省 35%；这里暂未计入传递码表等附加信息。

课程连接·编码和外排使用同一棵树

哈夫曼叶权可表示字符频数，也可表示外部归并段长。每次参加合并使该叶再负担一次权值，故总合并长度等于 WPL；第 8 章的最佳归并由此得到。

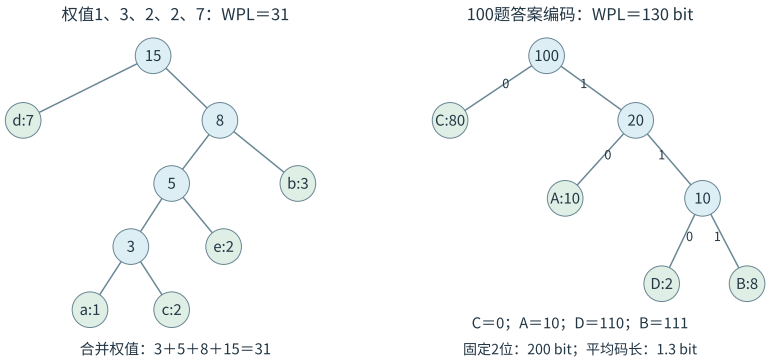


Figure 5.6: 两组王道课件数据的哈夫曼树、合并权值与码字

例如 `CAAABD` 编码为 `0|10|10|10|111|110`。沿树由根读取比特，到叶即输出一个字符再回根，就能解出原串；每个字符在叶上，因此完整码字不会是另一完整码字的前缀。

5.10.3 例 5-6　已占用的码字怎样限制剩余容量

例 5.10.2 (配套案例)

〖5.5-004〗哈夫曼编码长度不超过 4，已有两个字符编码为 1 和 01，还最多能对几个字符编码?

A. 2　B. 3　C. 4　D. 5

解析。以 1 开头的整棵子树已被码字 1 占用，以 01 开头的子树也被占用。剩余码字只能位于 00 开头的子树。长度至多 4，00 之后最多还放 2 位，最多形成 `0000`、`0001`、`0010`、`0011` 四个叶，答案为 **C**。使用较短的剩余码会占用其后代位置，不会增加字符总数。

推广到 m 叉最优归并/编码时，每次合并最小的 m 个权值。若叶数不满足满 m 叉树的条件 $(n-1) \bmod (m-1) = 0$，可补最少的零权虚叶。补齐后的叶数记 n'，内部结点数为 $(n'-1)/(m-1) = \lceil (n-1)/(m-1) \rceil$；不能直接把二叉的 “内部结点 $n-1$” 照搬。这里内部结点都按 m 叉合并计算，零权虚叶不代表额外字符。第 8 章最佳归并树会再次使用这项结构关系。

把哈夫曼的贪心选择写成可重复步骤

候选森林初始为各带权叶；重复取两个最小权根，创建权值为两者之和的新根，并将它放回候选集合。每轮候选数减少 1，故 n 个符号需 $n-1$ 轮。优先队列实现总时间 $O(n \log n)$，树与候选存储 $O(n)$；单符号编码的码长需另作约定。

```
huffman(weights):
    Q=以权值为键的小根优先队列
    while size(Q)>1:
        a=extractMin(Q); b=extractMin(Q)
        z=new Node(weight=a.weight+b.weight,left=a,right=b)
        insert(Q,z)
    return Q唯一根（若原输入为空则返回空）
```

不变量·压缩路径不压缩集合

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

5.11 并查集：用根结点代表一个集合

并查集维护若干互不相交的集合，主要支持两个操作：`find(x)` 查 x 所在集合的代表元，`union(x,y)` 合并两个集合。它不要求维护集合内部元素的顺序。用数组 `parent` 表示一片双亲森林，根存负数表示集合规模，非根存双亲下标。

例如初始 6 个单元素集合：`parent=[-1,-1,-1,-1,-1,-1]`。合并 0 与 1 后可记 `parent[0]=-2,parent[1]=0`；再合并 2 与 3 得 `parent[2]=-2,parent[3]=2`；把这两集合合并时，把根 2 接到根 0，根 0 规模记 -4。查找 3 时沿 $3\to2\to0$，不能把直接双亲 2 误当作最终代表。

按规模合并让较小树根接到较大树根，避免不断形成长链；**路径压缩**在查找后把沿途结点直接连到根。路径压缩不改变集合划分，只改变表示。两项同时使用时，连续操作的摊还复杂度极低，可用反阿克曼函数表示；单次查找仍须按当时实际路径执行，不能声称每次严格只访问一个结点。

```
void Initial(int S[], int n) {
    for (int i=0; i<n; ++i) S[i]=-1;
}
int Find(int S[], int x) {
    while (S[x]>=0) x=S[x];
    return x;
}
void Union(int S[], int Root1, int Root2) {
    if (Root1==Root2) return;
    S[Root2]=Root1;
}
```

这里的 `Union` 参数是两个**根下标**，调用前应先 `Find`。基础版本中根的负值只作根标志，合并时不维护规模。若使用按规模优化，则根负值必须始终等于负的结点总数，两种实现不能交叉调用。

```cpp
void UnionBySize(int S[], int Root1, int Root2) {
    if (Root1==Root2) return;
    if (S[Root2]>S[Root1]) {
        S[Root1]+=S[Root2];
        S[Root2]=Root1;
    } else {
        S[Root2]+=S[Root1];
        S[Root1]=Root2;
    }
}
int FindCompress(int S[], int x) {
    int root=x;
    while (S[root]>=0) root=S[root];
    while (x!=root) {
        int t=S[x];
        S[x]=root;
        x=t;
    }
    return root;
}
```

不变量·压缩路径不压缩集合

路径压缩把沿途结点直接挂根，保留每个元素最终代表及集合划分。根的负规模只在合并两个不同根时累加；重复合并同一集合不能再增加规模。

按规模合并使较小树结点每上升一层，其所属集合规模至少加倍，树高至多 $\lfloor \log_2 n \rfloor + 1$。路径压缩和按规模合并同时使用时，$m$ 次操作的总开销可写为 $O(m\alpha(n))$，这是摊还界；不能把它改说成任意一棵当前树高度严格等于 $\alpha(n)$。

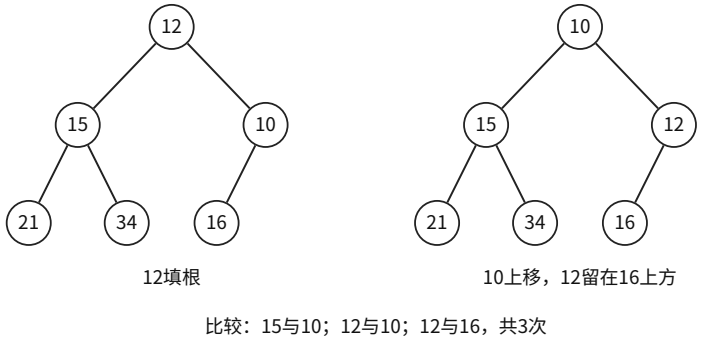
Kruskal 选边时，若两个端点代表元相同，加边会在现有森林中形成环；代表元不同才合并。这里代表的是当前已选边连成的分量，不是原图预先固定的一个编号。

5.12 堆的结构、调整与优先队列

二叉堆同时满足完全二叉树的形状和堆序。大根堆中每个双亲关键字不小于孩子，小根堆中不大于孩子。于是根保存全局最大值或最小值；兄弟间没有规定，整个数组也不必有序。堆中沿根到叶的关键字单调，二叉搜索树则按左右子树区间有序，二者不是同一个约束。

本书堆用下标从 1 开始的数组。小根堆的最大值一定可以在叶中找到，叶下标为 $\lfloor n/2 \rfloor + 1 \sim n$；不能确定是哪一个叶。非根结点若比所有后代都小，仍不保证它比别的子树中的元素小。

插入时先放到数组末尾，保证完全性，再与双亲比较并向上调整。删除根时，用末元素填根、缩小堆范围，再从根向下调整：选择较优孩子，与待调整的值比较，必要时交换并继续。每次最多跨越树高，时间 $O(\log n)$。原有左右子树必须已是堆，下滤才只需沿一条路径。



错项排除·堆序没有兄弟顺序

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。

Figure 5.7: 小根堆删根后，用末元素 12 填根，再与更小孩子 10 比较并下滤

建堆从最后一个非叶结点 $\lfloor n/2 \rfloor$ 开始逆序下滤。叶子本身已是堆，靠下的内部结点移动距离短；按各高度结点数求和，建堆总时间为 $O(n)$，不是把每个结点都按最坏树高估成 $O(n \log n)$。逐个插入也能建堆，但对应另一种过程和复杂度上界。第 8 章将用同一调整规则执行完整堆排序。

5.12.1 堆下滤的代码与不变式

以下大根堆采用 1 起始数组。调用前，左右子树各自满足堆序，只可能根位置过小；每轮把较大孩子提升，尚未解决的缺口沿一条链向下移动。

```
void HeadAdjust(int A[], int k, int len) {
    A[0]=A[k];
    for (int i=2*k; i<=len; i*=2) {
        if (i<len && A[i]<A[i+1]) ++i;
        if (A[0]>=A[i]) break;
        A[k]=A[i]; k=i;
    }
    A[k]=A[0];
}
```

向下每轮至少下降一层，最坏 $O(\log n)$，辅助 $O(1)$。只比较候选与左孩子会漏掉更大的右孩子；`i<len` 是检查右孩子确实存在。空堆不应调用此函数，外部循环从 $n/2$ 开始可自然跳过 $n = 0, 1$ 的情况。

结构	约束保存什么	操作与判断边界
普通二叉树	左右位置有别	遍历还原需序列与身份条件
完全二叉树	编号占据前 n 个位置	适合数组；只排除右孩子孤立还不够
二叉堆	完全形状及双亲孩子间堆序	根为最值；兄弟不保证有序
BST（第 7 章）	全部左右子树的关键字区间	中序有序；不保证完全或平衡
哈夫曼树	叶权下的 WPL 最小	通常不是完全树；编码可不唯一

5.13 主动回忆自测

以下为自拟补充，针对表示与前置条件。

1. 一棵二叉树有 8 个度为 2 的结点、3 个度为 1 的结点，共多少结点？它是否一定为完全二叉树？
2. 一棵二叉树的先序和中序完全相同，结点值互异，所有非叶结点的孩子位置有什么限制？
3. 并查集中 `parent[4]=2,parent[2]=0,parent[0]=-5`，执行带路径压缩的 `find(4)` 后，哪些关系改变？
4. 关键字互不相同的大根堆中，第二大元素一定在哪些可能位置？
5. 为什么一棵 n 结点二叉链表有 $n+1$ 个空孩子域？加上双亲域后怎样变化？
6. 普通树的后根遍历转换后对应二叉树哪一种遍历？为什么不是后序？
7. 权值 3、4、5 合并构造哈夫曼树，WPL 是多少？编码一定唯一吗？
8. 层序遍历与递归深度优先遍历分别由什么结构保存未处理状态？最坏辅助空间是否一定相同？

答案。

1. 由二叉树的性质 $n_0 = n_2 + 1$，得叶结点数 $n_0 = 8 + 1 = 9$，所以总结点数为 $9 + 3 + 8 = 20$。这棵树**不是完全二叉树**：完全二叉树至多有 1 个度为 1 的结点，而题中有 3 个，不满足这一必要条件。
2. 只能有右孩子；若存在左子树，中序会先于根访问它，与先序矛盾。
3. 返回 0，`parent[4]` 改为 0，2 仍指向 0，根规模不变。
4. 根的孩子中；若某更深结点是第二大，其双亲严格大于它，除根外还会出现一个更大的结点，产生矛盾。存在相同值时，须先明确第二大是否按不同值计数，不能原样沿用结论。
5. $2n$ 个孩子域中 $n-1$ 个非空，余 $n+1$；再加 n 个双亲域，除根外 $n-1$ 个非空，总空域 $n+2$。
6. 对应中序：左链递入孩子森林、右链枚举兄弟，全部孩子完成才访问本根；右孩子在转换语义中不是原树子女，不能按原二叉后序套用。
7. 先 3+4=7，再 5+7=12，WPL 为 7+12=19；左右 0/1 交换可改变编码而不改变 WPL。
8. 前者用队列、后者用调用栈；队列与宽度有关，调用栈与高度有关，形态不同可相差很大。

错项排除·堆序没有兄弟顺序

小根堆的根最小，两个孩子之间却没有规定。删除根时必须先选更优孩子，再和候选比较；只因左孩子更早出现在数组，就总往左走，会破坏右侧堆序。